

Testing

Many things can go wrong in software, from confusing interfaces to crashes. Developers follow a multi-step testing process to cover as many bases as possible.

SEE ALSO

◀ 114–115 Software errors

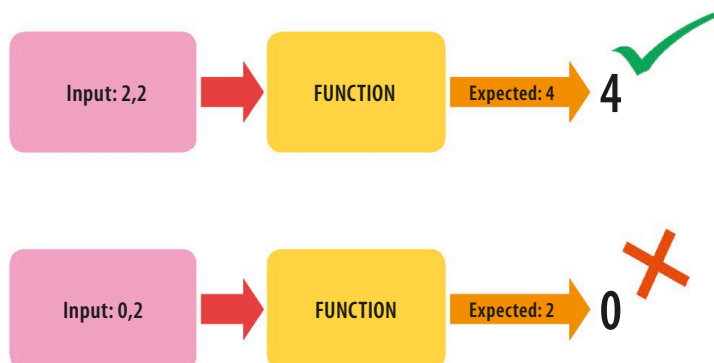
◀ 168–169 Developing and designing

Maintenance and support

174–175 ▶

Unit tests

These target a small block of code, usually a function, and make sure that it works under all conditions. A program to display tickets in a printable format will first test for a single ticket, then for two tickets, and then for 10,000 tickets. Each typical case as well as edge case is covered. Edge cases are scenarios that are unlikely but must still be checked to prevent future problems.



LINGO

Functional vs Non-functional

A functional test checks if the code works. A non-functional test checks how well the code works. While functional tests are easier to write, they're not adequate to prove that software fulfils all of its requirements.



△ What testers write

A test is a special mini-script. Developers write unit tests for each segment of the program, specifying inputs and checking if the real output matches the expected output. Tests are often written in the same language as the main software.

△ Mock data

Certain functions require specially structured data. If the data is user-provided then it's not available in a test case scenario, so developers create mock data to mimic what a user might enter. Crash test dummies are a real-life example.

Integration testing

Unit tests work with isolated pieces of code. New problems sometimes crop up when those pieces are fitted together. At other times, gaps are discovered in the program's functionality. Integration tests are longer, more complex test cases that combine different sections of code and make sure that everything works as expected. The goal is to model how the software will actually be used.

▷ Simulation

To test a virtual game of cards, you might create mock players and then simulate them to play a few fake rounds.

